INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES RENNES

Web2
Planning and Deliverables

4INFO – Computer Science Department

Arnaud Blouin
arnaud.blouin@irisa.fr

Contents

1 Project Team **2**

2 Project management: hybrid Agile **2**

3 Sprint #1 **3**

4 Sprint #2 **4**

5 Sprint #3 **4**

6 Next sprints **5**

7 Marking **5**

The practical sessions are not enough to complete the project. You must work in pairs aside the sessions. Start as soon as possible. Late rush is bound to fail. Use Teams to ask questions, not email.

1 Project Team

You have to follow these rules to create your project team:

- **Two students per project team**. If your group has an odd number of students, one team can be composed of three students. For this group, the evaluation will be more strict.
- **A project team must have at max: one student retaking ("*redoublant*") or a newcomer ("*nouvel arrivant*")**. So, it is not possible to have a team composed of two retaking students, or of one retaking student and one newcomer, or of two newcomers.

2 Project management: hybrid Agile

To complete the project we will follow a hybrid Agile management:

- you will release several versions of your project (every month or every two weeks approximately) for developing the game incrementally;
- you will write user stories (business analyst job) to capture client (me) needs;
- you will use git and gitlab-ci to handle issues, milestones, continuous integration.

User story. Details one feature of the system. Example:

En tant que joueur,
je veux visualiser mon score pendant la partie
afin d'améliorer l'expérience utilisateur.
A user story is composed of three elements: the scope (*en tant que ...*); the goal (*je veux...*); the origin of the need (*afin de ...*).

Acceptance criteria. Associated to one user story, an acceptance criteria details how to implement it to be accepted by the client. The goal of acceptance tests is the precise the

different possible scenarios of each user stories. Example:
Pendant une partie, le score sera affiché et mis à jour sur la droite de la grille.

Sprint. A sprint is a (short) period of development, during which a given number of usr stories are considered. A sprint results in a deliverable (a release).

Issue. An issue (or ticket) refers to issue tracking of your project on gitlab. You will open issues either to refer to one user story, or to detail a bug to fix.

Backlog. A backlog is the set of issues. There exists two kinds of backlogs: the product backlog refers to all the issues of your project; a sprint backlog refers to selected issues targeted for this sprint.
How to create a backlog using gitlab:

- on gitlab in the *issues* view, open 'issues';
- on gitlab in the *issues* view, create a milestone with as name *DX* (X being the number of the sprint, DX = delivrable X). This gitlab milestone will be your backlog for this sprint. Add the issues this milestone.
- assign one or several persons to each issue.

Retrospective. After the end of each sprint, engineers discuss together about the past sprint (to discuss problems, the next sprint).

Requirement document. This is not an agile concept. You will start by writing user stories based on a video and discussions with me to understand what I expect (agile process).
But I want you to do the right things (and the things right), so that after that I will provide you a requirement document that details all I expect for the project. You will write user stories in gitlab issues based on those requirements.
This is not an fully Agile process since Agile implies that the requirements are not (all) known prior the development, which is not our case.

3 Sprint #1

The very first step consists in writing user stories and related acceptance criteria (in natural text). During the first practical session (TP) you will watch the video and ask me about the features I want for the software system.
For this sprint the backlog (we do not use gitlab yet for backlog) is composed of:

- user stories to write in French, as issues in Gitlab (How many? As far as necessary...);
- write one or several acceptance criteria for each user story (either as other issues or using a list in the description of the user story);
- push the base code we provide on a your gitlab repository (https://gitlab.insa-rennes.fr);
- configure the CI (continuous integration) of your gitlab project to build the front-end and the back-end on each commit (see the documentation I provide on Moodle);
- add a git tag *D1*.

The deadline for this deliverable is 20/09/24 (end of the day, tentative).

4 Sprint #2

I will now give you my requirement document that precisely describes each requirement I want for the game. You can compare your user stories and their acceptance tests to check whether you miss (and misunderstand) things.
For this sprint the backlog (open issues on gitlab) is:

- Front-end: display the grid. Do not use the back-end, no data model. Just display the map with hard-coded values.
- Design a UML class diagram of the data model of the application (image to put the folder *game-doc*). You can use https://www.planttext.com/ or https://mermaid-js.github.io/mermaid-live-editor/

  The first goal of your UML models is not to generate Java or TypeScript code, but to brainstorm about classes to have, their relations, attributes, etc. Do sketches first. Do real UML diagrams at the end of the sprint.

  Do not consider the Angular view (Angular components) in this class diagram. Consider Angular services. Do not consider the back-end and its REST routes. Note that you may have some similar interfaces both on the back and front ends. You will have to use some design patterns in your design.
- Add a git tag *D2*.
- The CI must be successfully pass on this tag.

The deadline for this deliverable is 11/10/24 (end of the day, tentative).

5 Sprint #3

For this sprint the backlog is:

- Front-end: display the grid with the data model. Do not use the back-end, but fake data. Data model = code your class diagram. Add an Angular service that contains the current game, so that your components can access this last to display the grid.
- Class diagram of the back-end. This diagram will share for sure elements of the data model class diagram. The goal is to brainstorm about how to store data (and which data, in which format). Integrate in this diagram the REST controller(s) that will implement the REST routes.
- Design the REST API that the back-end will provide to the front-end. For example: `GET /game/foo/bar` => JSON instance of the class Foo with code OK (or code NOT_FOUND if does not exist). Put the routes in a markdown files in *game-doc*.
- Add a git tag *D3*.
- The CI must be successfully pass on this tag

The deadline for this deliverable is 01/11/24 (end of the day, tentative).

6 Next sprints

You will then manage your own backlog (using gitlab) for each of the next sprints to provide each two weeks (15/11/24, 29/11/24, 13/12/24, 10/01/25, tentatives).
During a sprint, you have to:

- on gitlab in the *issues* view, add 'issues' you will do for the sprint;
- manage you won backlog (gitlab milestone);
- Add a corresponding git (D4, D5, D6, D7);
- The CI must be successfully pass on each tag.

Of course you can postpone issues to the next sprint.

7 Marking

The following marks (that may change) will be used:

1. The modelling elements (user stories, class diagram, etc.) $\approx$ 5 pts;
2. The final deliverable (the one I will test for real) $\approx$ 12 pts ;
3. The CI, the way you managed your project, etc. $\approx$ 3 pts.

How to lose points (no limited to):

- Deadlines not respected (even some minutes).
- Do not provide what each deliverable must contain (no backlog, no/few issue).
- No Gitlab
- No CI
- CI does not build
- *arnaud.blouin* is not maintainer on your git project
- Project somewhere else than on the INSA gitlab (e.g., on Github)
- Do not use gitlab issues and milestones
- Plagiarism (a lot of points)